B.10 — Implementazione della firma del verbale

Guida step-by-step all'architettura e al codice

Progetto ICMVerbali

2026-05-13

Introduzione

Questo documento racconta — passo dopo passo, in modo discorsivo — come abbiamo implementato la **firma del CSE** sul verbale di sopralluogo: dalla pressione del bottone "Salva e firma" nella UI fino al salvataggio del verbale firmato nel database, con il PNG della firma scritto sul filesystem e il numero progressivo annuale assegnato.

L'obiettivo è duplice: (1) chi entrerà nel progetto da junior trova qui un percorso ordinato per capire *cosa* abbiamo costruito e *perché*; (2) chi rivisita il codice fra mesi ritrova qui le decisioni di design senza dover ricostruire il ragionamento partendo dai diff.

Il livello tecnico assunto è quello di un developer .NET con esperienza media: si dà per nota la sintassi di base di C# e Blazor, ma ogni concetto specifico (JS interop, lock pessimistico, race condition, transazione) viene esplicitato dove serve.

---

1. Visione d'insieme

Cosa abbiamo costruito

Una "firma elettronica semplice" inline. L'utente CSE, dopo aver compilato un verbale di sopralluogo (10 step di wizard), clicca **"Salva e firma"** sull'ultima pagina di riepilogo. Si apre un **dialog modale** con un riquadro su cui disegnare la firma con dito o stilo, un campo testo pre-compilato con il nome del CSE (editabile), e tre bottoni: *Pulisci / Annulla / Conferma firma*.

Premendo "Conferma firma" la firma viene:

1. **trasformata in PNG** lato browser (`canvas.toDataURL('image/png')`)
2. **inviata al server** via Blazor Server (SignalR)
3. **validata**: tutte le anagrafiche obbligatorie devono essere presenti, esito e meteo devono essere selezionati
4. **salvata sul disco** come file PNG sotto `App_Data/uploads/firme/{verbaleId}/cse.png`
5. **registrata nel DB** in una transazione che: assegna un numero progressivo annuale al verbale, crea la riga `Firma`, sposta lo stato del verbale da `Bozza` a `FirmatoCse`, scrive una riga di audit

L'utente vede uno snackbar "Verbale firmato. Numero 4/2026." e viene riportato alla home, dove il verbale ora compare nella lista "verbali del giorno" con etichetta "FirmatoCse".

I tre livelli architetturali coinvolti

L'app segue un'architettura **layered N-tier** classica:

```
+-------------------------------------+
|  Presentation (Blazor)              |  <-- WizardStep10Riepilogo.razor
|                                     |      SignaturePadDialog.razor
+-------------------------------------+
|  Application (Manager)              |  <-- VerbaleManager.FirmaCseAsync
|                                     |      VerbaleValidator.PuoFirmare
+-------------------------------------+
|  Data (Repository, Dapper)          |  <-- VerbaleRepository.FirmaCseAsync
|                                     |      FirmaRepository (read-only)
+-------------------------------------+
                  |
                  v
            +---------+      +--------------+
            | SQL DB  |      | Filesystem   |
            |         |      | (PNG firma)  |
            +---------+      +--------------+
```

Più due servizi orizzontali:

- `IFirmaStorageService` astrae la scrittura del PNG (oggi su filesystem, in futuro spostabile su Azure Blob senza toccare i manager)

- **JS interop wrapper** (`wwwroot/js/signatureInterop.js`) astrae l'uso della libreria JavaScript `signature_pad`

2. File creati e modificati

In totale **12 file nuovi** + **7 file modificati**. Ecco la mappa.

File creati

File	Zona	Cosa fa
`wwwroot/lib/signature_pad/signature_pad.umd.min.js`	Asset	Libreria JS terza parte (MIT, 12.5 KB) che disegna la firma sul canvas
`wwwroot/js/signatureInterop.js`	Asset	Modulo ES che esporta init/clear/getDataUrl/dispose. Astrae signature_pad da Blazor
`Storage/IFirmaStorageService.cs`	Storage	Interfaccia per salvare/leggere PNG firma
`Storage/LocalFirmaStorageService.cs`	Storage	Implementazione filesystem dell'interfaccia sopra
`Repositories/Interfaces/IFirmaRepository.cs`	Data	Letture firme (per servire il PNG via API)
`Repositories/FirmaRepository.cs`	Data	Implementazione Dapper di sopra
`Managers/VerbaleValidator.cs`	Application	Helper statico: "questo verbale può essere firmato?"
`Managers/VerbaleNonFirmabileException.cs`	Application	Eccezione tipizzata con lista errori di validazione
`Components/Shared/SignaturePadDialog.razor`	UI	Il dialog con canvas + bottoni

File modificati

File	Cosa è cambiato
`Entities/Enums/EventoAuditTipo.cs`	Aggiunto `Firma = 3`
`Repositories/Interfaces/IVerbaleRepository.cs`	Aggiunto `FirmaCseAsync` + record `FirmaCseResult`
`Repositories/VerbaleRepository.cs`	Implementazione di `FirmaCseAsync` (transazione cross-tabella)
`Managers/Interfaces/IVerbaleManager.cs`	Aggiunto `FirmaCseAsync` con eccezione tipizzata
`Managers/VerbaleManager.cs`	Iniezione `IFirmaStorageService` + `TimeProvider`; metodo `FirmaCseAsync`
`Components/Pages/Verbali/WizardStep10Riepilogo.razor`	Bottone “Salva e firma” ora apre il dialog (era stub)
`Program.cs`	Registrazione dei nuovi servizi nel DI; nuovo endpoint API per servire il PNG

3. Step by step: come ogni pezzo è stato costruito

In questa sezione percorriamo l'implementazione in ordine *bottom-up*: prima i mattoni più bassi (asset JS, storage), poi i livelli intermedi (repository, manager), infine la UI che li orchestra. È l'ordine in cui sono stati scritti durante lo sviluppo, e anche l'ordine in cui hanno senso le letture: ogni layer dipende solo da quelli sotto.

3.1 La libreria JavaScript

La firma "live" su canvas richiede una libreria che traduca i movimenti del puntatore (dito, mouse, stilo) in una linea pulita disegnata sul canvas, con smoothing variabile in base alla velocità del tratto e supporto per pressione e tilt dove disponibili. Implementare bene questa logica costa qualche centinaio di righe di JavaScript: ci affidiamo a `signature_pad`, una libreria MIT da ~12 KB che fa esattamente questo.

L'asset minificato è scaricato una volta dalla CDN ufficiale e copiato in `wwwroot/lib/signature_pad/signature_pad.umd.min.js`. È versionato (5.0.4) nel repository: non c'è dipendenza runtime su Internet, l'app gira offline.

3.2 Il wrapper JS interop

Blazor Server non parla JavaScript direttamente: parla via **JS interop**, cioè invoca funzioni JS dal lato C# attraverso il servizio `IJSRuntime`. Per comodità ed isolamento abbiamo creato un piccolo modulo *ES module* in `wwwroot/js/signatureInterop.js` che esporta quattro funzioni:

```
export async function init(canvasId, options) { ... }
export function clear(canvasId) { ... }
export function isEmpty(canvasId) { ... }
export function getDataUrl(canvasId, mime) { ... }
export function dispose(canvasId) { ... }
```

Tre dettagli importanti dentro questo modulo:

(a) Lazy load della libreria. `signature_pad.umd.min.js` viene caricato solo la prima volta che apriamo il dialog di firma, non al boot dell'app. Una `Promise` cache evita che chiamate concorrenti di `init` carichino lo script due volte.

(b) WeakMap canvas → SignaturePad. Ogni dialog ha un suo canvas con un `id` univoco generato in C# (`sig-{guid}`). La `WeakMap` collega l'elemento DOM all'istanza JS senza tenere riferimenti forti — quando il dialog viene smontato e il canvas rimosso dal DOM, anche l'istanza JS diventa garbage collectible.

(c) Resize per HiDPI. I display ad alta densità (Retina, schermi 4K) mostrerebbero una firma "scalettata" se il canvas non viene scalato per `devicePixelRatio`. La funzione `resizeCanvas` ridimensiona il canvas alla dimensione fisica reale prima di passarlo a `signature_pad`.

3.3 Lo storage del PNG

La firma, alla fine del processo, è un'immagine PNG che deve essere salvata da qualche parte. Abbiamo seguito lo stesso pattern già adottato per le foto del verbale in B.9: un'interfaccia astratta `IFirmaStorageService` e un'implementazione concreta `LocalFirmaStorageService` che scrive su filesystem. Domani, se servirà spostare lo storage su Azure Blob o S3, basterà scrivere una seconda implementazione e cambiare la registrazione nel DI: nessun manager o controller dovrà essere toccato.

L'interfaccia:

```
public interface IFirmaStorageService
{
    Task<FirmaStorageResult> SalvaAsync(
        Guid verbaleId, TipoFirmatario tipo, byte[] pngBytes, CancellationToken ct =
        default);
    Task<Stream> ApriLetturaAsync(string filePathRelativo, CancellationToken ct = default);
    Task EliminaAsync(string filePathRelativo, CancellationToken ct = default);
}

public sealed record FirmaStorageResult(string FilePathRelativo, long Bytes, string
        Sha256Hex);
```

La convenzione di path è semplice: `firme/{verbaleId}/{cse|impresa}.png`. Una sola firma per coppia (verbale, tipo): sovrascrivere è permesso (se per un errore correggibile la firma va rifatta) ma rimane comunque blindato dall'UNIQUE constraint sul DB.

L'implementazione concreta scrive i byte ricevuti come sono — il PNG arriva già renderizzato dal canvas, nessun resize o re-encode è necessario — e calcola un hash SHA-256 che viene ritornato al chiamante. L'hash non viene salvato nel DB in B.10 (è disponibile per audit futuro di integrità, ad esempio se in futuro vorremo verificare che il file su disco non sia stato manomesso).

Un dettaglio di sicurezza: il metodo privato `ResolveAndValidate` impedisce *path traversal*. Se per qualche motivo arrivasse un path come `../../etc/passwd`, l'app rifiuterebbe l'operazione invece di leggere/scrivere fuori dalla cartella `App_Data/uploads/`.

3.4 Il repository

Il `FirmaRepository` è volutamente piccolo: contiene solo letture (`GetByVerbaleAsync`, `GetByVerbaleAndTipoAsync`). Le scritture infatti sono accoppiate alla transizione di stato del Verbale e vivono altrove — vedi il prossimo punto.

La parte interessante è invece il nuovo metodo nel `VerbaleRepository` esistente:

```
public async Task<FirmaCseResult> FirmaCseAsync(
    Guid verbaleId, int anno,
    string nomeFirmatario, DateOnly dataFirma,
    string immagineFirmaPath, Guid utenteId,
    CancellationToken ct = default)
```

Questo metodo è l'unico punto del codice che scrive una firma sul DB, e lo fa **dentro un'unica transazione** che coinvolge tre tabelle:

1. `Verbale` — verifica con lock che lo stato sia `Bozza`, calcola il prossimo `Numero` per l'anno corrente, aggiorna `Stato = FirmatoCse`, `Numero`, `Anno`, `UpdatedAt`.
2. `Firma` — inserisce una riga `Tipo = Cse`, con `NomeFirmatario`, `DataFirma`, `ImmagineFirmaPath`.
3. `VerbaleAudit` — inserisce una riga con `EventoTipo = Firma` e `Note = "Firma CSE: {nome}"`.

Se uno qualsiasi di questi tre step fallisce, l'intera transazione viene annullata (rollback) e il DB resta esattamente come prima.

Il lock pessimistico per la numerazione progressiva

Il problema dei numeri progressivi è classico: due utenti che firmano due verbali diversi nello stesso istante non devono ottenere lo stesso Numero. La query è semplice:

```
SELECT ISNULL(MAX(Numero), 0)
FROM dbo.Verbale WITH (UPDLOCK, HOLDLOCK)
WHERE Anno = @Anno AND Numero IS NOT NULL;
```

Le due *hint* `UPDLOCK` e `HOLDLOCK` dicono a SQL Server: "tieni un lock di scrittura sulle righe lette finché la transazione non finisce". Risultato: se due transazioni partono insieme, SQL ne serializza una, l'altra aspetta. La prima legge `MAX = 3`, scrive `Numero = 4`, commita. La seconda — sbloccata — legge `MAX = 4`, scrive `Numero = 5`. Nessuno ottiene 4 due volte.

Come **rete di sicurezza finale**, già dalla migration 001 c'è un `UNIQUE` filtrato sulla coppia `(Anno, Numero)`:

```
CREATE UNIQUE INDEX UQ_Verbale_Anno_Numero
    ON dbo.Verbale (Anno, Numero)
    WHERE Anno IS NOT NULL AND Numero IS NOT NULL;
```

Anche se per un bug il lock dovesse fallire, il DB rifiuta proprio la doppia firma con lo stesso numero. Sicurezza a due livelli.

3.5 Il validator

Una *bozza* può essere salvata anche se compilata solo parzialmente: questa è l'intera UX del wizard, dove l'utente può cambiare step e perdere/recuperare il lavoro in qualunque momento. Ma per **firmare** un verbale, alcune cose devono essere presenti. La domanda "questo verbale è firmabile?" è esplicita in un piccolo helper statico:

```
public static class VerbaleValidator
{
    public static VerbaleValidationResult PuoFirmare(Verbale verbale)
    {
        var errori = new List<string>();
        if (verbale.CantiereId == Guid.Empty) errori.Add("Cantiere non impostato.");
        // ... 6 altre anagrafiche
        if (verbale.Esito is null)  errori.Add("Esito non selezionato (sez. 2).");
        if (verbale.Meteo is null)  errori.Add("Condizione meteo non selezionata (sez.
         2).");
        return new VerbaleValidationResult(errori.Count == 0, errori);
    }
}
```

Il vincolo è **deliberatamente minimo**: anagrafiche complete + esito + meteo. Niente vincoli su prescrizioni, foto, temperatura, interferenze. Il razionale: il valore legale del verbale è la firma del CSE; le checklist e le foto sono materiale probatorio opzionale; mettere vincoli più stringenti rischia di bloccare casi legittimi in cantiere senza beneficio normativo.

Quando la validazione fallisce, il manager lancia una **eccezione tipizzata** `VerbaleNonFirmabileException` con dentro la lista degli errori. Questo permette alla UI di mostrare un messaggio puntuale invece di un generico "operazione fallita":

```
public sealed class VerbaleNonFirmabileException : Exception
{
    public IReadOnlyList<string> Errori { get; }
    public VerbaleNonFirmabileException(IReadOnlyList<string> errori)
        : base("Il verbale non puo' essere firmato: validazione fallita.")
    {
        Errori = errori;
    }
}
```

3.6 Il manager

Il `VerbaleManager` è l'orchestratore: prende l'input dell'utente, lo valida, decide cosa fare, e chiama i layer sotto. Il metodo nuovo è `FirmaCseAsync`:

```
public async Task<FirmaCseResult> FirmaCseAsync(
    Guid verbaleId, string nomeFirmatario, byte[] pngBytes,
    Guid utenteId, CancellationToken ct = default)
{
    // 1. Carica e verifica stato
    var verbale = await _repo.GetByIdAsync(verbaleId, ct)
        ?? throw new InvalidOperationException(...);
    if (verbale.Stato != StatoVerbale.Bozza)
        throw new InvalidOperationException(...);

    // 2. Validazione hard
    var validation = VerbaleValidator.PuoFirmare(verbale);
    if (!validation.IsValid)
        throw new VerbaleNonFirmabileException(validation.Errori);

    // 3. Salva PNG su disco
    var storageResult = await _firmaStorage.SalvaAsync(
        verbaleId, TipoFirmatario.Cse, pngBytes, ct);

    // 4. Calcola anno corrente
    var now = _clock.GetUtcNow().UtcDateTime;
    var anno = now.Year;
    var dataFirma = DateOnly.FromDateTime(now);

    // 5. Transazione DB
    return await _repo.FirmaCseAsync(
        verbaleId, anno, nomeFirmatario.Trim(), dataFirma,
        storageResult.FilePathRelativo, utenteId, ct);
}
```

Due punti meritano commento.

L'ordine "PNG prima del DB". Salviamo l'immagine PRIMA di toccare il DB. Se la scrittura su disco fallisce (disco pieno, permessi mancanti), niente verrà scritto in DB: il verbale rimane in `Bozza`, l'utente vede l'errore. Se invece la transazione DB fallisce DOPO che il PNG è stato salvato, il file resta orfano sul disco. Abbiamo accettato questo trade-off (orfani benigni, garbage-collectabili) perché l'alternativa — salvare nel DB i bytes del PNG come `varbinary` — gonfierebbe il backup del DB e renderebbe lente le query. Stesso pattern già usato per le foto in B.9.

`TimeProvider` invece di `DateTime.UtcNow`. Iniettiamo `TimeProvider` come dipendenza invece di chiamare direttamente `DateTime.UtcNow`. In produzione passa `TimeProvider.System`, che è equivalente. Ma nei test possiamo passare un `FakeTimeProvider` per simulare scenari "firmato il 31 dicembre alle 23:59 ma transazione committata il 1° gennaio alle 00:00:01" — utile per verificare la regola "il numero progressivo si resetta al cambio d'anno".

3.7 L'endpoint API

La firma salvata deve poter essere mostrata da qualche parte (in futuro: nel dettaglio del verbale, in un PDF generato). Per servire l'immagine abbiamo aggiunto un endpoint Minimal API in `Program.cs`:

```
app.MapGet("/api/firme/{verbaleId:guid}/{tipo}", async (
    Guid verbaleId, string tipo,
    IFirmaRepository repo, IFirmaStorageService storage,
    CancellationToken ct) =>
{
    if (!Enum.TryParse<TipoFirmatario>(tipo, ignoreCase: true, out var tipoEnum))
    {
        if (string.Equals(tipo, "impresa", StringComparison.OrdinalIgnoreCase))
            tipoEnum = TipoFirmatario.ImpresaAppaltatrice;
        else return Results.NotFound();
    }
    var firma = await repo.GetByVerbaleAndTipoAsync(verbaleId, tipoEnum, ct);
    if (firma is null || string.IsNullOrEmpty(firma.ImmagineFirmaPath))
        return Results.NotFound();
    var stream = await storage.ApriLetturaAsync(firma.ImmagineFirmaPath, ct);
    return Results.File(stream, "image/png", enableRangeProcessing: false);
}).RequireAuthorization();
```

Tre cose da notare:

- `RequireAuthorization()`: solo utenti autenticati possono richiedere il PNG. La nostra policy attuale è permissiva ("tutti gli utenti autenticati possono vedere tutto"), ma il punto di estensione è qui — basterà aggiungere una verifica "questo utente può vedere questo verbale" quando definiremo le visibility rules.
- **Doppia parsing del segmento** `tipo`: accettiamo sia la forma esatta dell'enum (`ImpresaAppaltatrice`) sia l'alias breve `impresa`, perché il primo è scomodo da scrivere a mano.
- **Niente range processing**: per le immagini di firma (~20 KB) il browser non ha bisogno di range request. Le foto, che sono più grandi, lo stesso. Lo abilitiamo se mai serviremo file molto grandi.

3.8 Il dialog di firma (UI)

Il `SignaturePadDialog.razor` è il componente che l'utente vede quando clicca "Salva e firma". È un dialog MudBlazor con tre parti: in alto un `MudTextField` per il nome firmatario (pre-compilato col nome del CSE dell'anagrafica, editabile); in mezzo il canvas; in fondo i tre bottoni *Pulisci / Annulla / Conferma firma*.

La struttura C# del dialog è scandita in tre momenti del ciclo di vita Blazor:

`OnInitialized`: legge il parametro `NomeFirmatarioSuggerito` e lo mette nel campo testo.

`OnAfterRenderAsync(firstRender)`: dopo che il DOM è stato montato, importa il modulo JS via `IJSRuntime.InvokeAsync<IJSObjectReference>("import", "/js/signatureInterop.js")` e chiama `init` per agganciare `signature_pad` al canvas:

```
_module = await JS.InvokeAsync<IJSObjectReference>("import", "/js/signatureInterop.js");
await _module.InvokeVoidAsync("init", _canvasId, new {
    backgroundColor = "rgb(255,255,255)",
    penColor = "rgb(0,0,0)",
});
```

`DisposeAsync`: alla chiusura del dialog rilascia il signature_pad lato JS (rimuove i listener) e disposa il `IJSObjectReference`.

Il flusso di conferma è quello atteso:

1. Estrai data URL dal canvas: `await _module.InvokeAsync<string?>("getDataUrl", _canvasId, "image/png")`
2. Strip del prefisso `` e decode in `byte[]`
3. Recupera l'`utenteId` dal claim `NameIdentifier`
4. Chiama `VerbaleManager.FirmaCseAsync(...)`
5. Se l'eccezione è `VerbaleNonFirmabileException`, mostra la lista errori in un `MudAlert Warning` **dentro il dialog**, lasciandolo aperto
6. Se l'eccezione è `InvalidOperationException`, mostra il messaggio in un `MudAlert Error`
7. Se va a buon fine, `Dialog.Close(DialogResult.Ok(result))` restituisce il `FirmaCseResult` al chiamante

3.9 Il wire-up nello step 10

Lo step 10 del wizard (riepilogo) aveva già il bottone "Salva e firma", ma era uno stub che mostrava solo uno snackbar dummy. L'abbiamo collegato al dialog:

```
private async Task HandleFirmaAsync()
{
    if (_signing) return;
    _signing = true;
    try
    {
        var parameters = new DialogParameters
        {
            [nameof(SignaturePadDialog.VerbaleId)] = Existing.Id,
            [nameof(SignaturePadDialog.NomeFirmatarioSuggerito)] = _cse?.Nominativo ??
         string.Empty,
        };
        var dialog = await DialogService.ShowAsync<SignaturePadDialog>("Firma CSE",
         parameters, options);
        var result = await dialog.Result;

        if (result is { Canceled: false, Data: FirmaCseResult firma })
        {
            Snackbar.Add($"Verbale firmato. Numero assegnato:
         {firma.NumeroAssegnato}/{firma.Anno}.",
                Severity.Success);
            Nav.NavigateTo("/");
        }
    }
    finally { _signing = false; }
}
```

Niente di sofisticato: passiamo i due parametri al dialog (l'`Id` del verbale e il nome suggerito del CSE), aspettiamo che si chiuda, e se il risultato è un `FirmaCseResult` mostriamo il numero assegnato e

torniamo alla home.

4. Il flusso end-to-end raccontato come un giro

Vediamo ora cosa succede, momento per momento, quando l'utente preme "Conferma firma" sul dialog. Questa è la sequenza che mette insieme tutti i pezzi descritti sopra.

Lato browser (0–50 ms):

1. Click sul bottone "Conferma firma" del dialog.
2. Il signature pad JS produce il data URL del canvas: `...` (~25 KB).
3. Blazor Server invia il valore al server attraverso il circuito SignalR aperto.

Lato server, codice del dialog (50–80 ms):

4. `HandleConfirmAsync` riceve il data URL, fa lo strip del prefisso e ottiene `byte[]`.
5. Estrae l'`utenteId` dai claim del cookie auth.
6. Chiama `VerbaleManager.FirmaCseAsync(verbaleId, nome, pngBytes, utenteId)`.

Lato server, manager (80–120 ms):

7. `_repo.GetByIdAsync(verbaleId)` legge il verbale dal DB.
8. Verifica `Stato == Bozza`.
9. `VerbaleValidator.PuoFirmare(verbale)` controlla anagrafiche + esito + meteo. Se KO, lancia `VerbaleNonFirmabileException` con la lista. Il dialog mostra il `MudAlert` di errore e l'utente può correggere (chiude il dialog, modifica gli step incompleti, riprova).
10. `_firmaStorage.SalvaAsync(verbaleId, Cse, pngBytes)` scrive il PNG su disco in `App_Data/uploads/firme/{verbaleId}/cse.png` (~25 KB).
11. Recupera l'anno corrente: `_clock.GetUtcNow().Year`.

Lato server, repository, dentro la transazione (120–180 ms):

12. `BEGIN TRANSACTION`.
13. `SELECT Stato FROM Verbale WITH (UPDLOCK, HOLDLOCK) WHERE Id = @Id` — lock acquisito.
14. `SELECT ISNULL(MAX(Numero), 0) FROM Verbale WITH (UPDLOCK, HOLDLOCK) WHERE Anno = @Anno` — supponiamo torni 3.
15. `INSERT INTO Firma (Id, VerbaleId, Tipo=Cse, NomeFirmatario, DataFirma, ImmagineFirmaPath)` — UQ_Firma_VerbaleId_Tipo passa (era la prima firma CSE).
16. `UPDATE Verbale SET Stato = FirmatoCse, Numero = 4, Anno = 2026, UpdatedAt = SYSUTCDATETIME() WHERE Id = @Id`.
17. `INSERT INTO VerbaleAudit (Id, VerbaleId, UtenteId, DataEvento, EventoTipo = Firma, Note = "Firma CSE: ...")`.
18. `COMMIT`.

Ritorno al manager e al dialog (180–200 ms):

19. Il manager riceve `FirmaCseResult(NumeroAssegnato = 4, Anno = 2026)` e lo restituisce al chiamante.
20. Il dialog fa `Dialog.Close(DialogResult.Ok(result))`.

Step 10 e UI finale (200–250 ms):

21. `HandleFirmaAsync` riceve il risultato.
22. Snackbar verde: “Verbale firmato. Numero assegnato: 4/2026.”
23. `Nav.NavigateTo("/")` — naviga alla home.
24. La home interroga il DB: il verbale ora compare nella lista del giorno con etichetta “FirmatoCse” e numero 4/2026.

Tempo totale percepito: ~250 ms su una rete locale, contenuto bene anche su 4G.

5. Decisioni di design

Durante l'implementazione abbiamo preso cinque decisioni che vale la pena documentare separatamente, perché incidono sulla forma del codice ma non si vedono guardando solo i diff.

5.1 Signature pad: JavaScript via interop, non NuGet

Avevamo tre opzioni: la libreria JS via interop, un wrapper NuGet (`BlazorSignaturePad`), o un canvas custom scritto interamente da noi. Abbiamo scelto la **prima**.

Razionale: aggiungere un NuGet ha un costo perpetuo (sicurezza, breaking change, dipendenze transitive); scrivere noi un canvas+pen handling è lavoro che non porta valore (è risolto bene da `signature_pad` da anni); JS interop è un pattern Blazor standard, semplice da capire e mantenere.

5.2 Dialog inline invece di nuovo step 11

Avremmo potuto aggiungere uno step 11 al wizard, dedicato alla firma. Abbiamo scelto invece un `MudDialog` aperto dallo step 10. Razionale: il canvas + JS interop sono "extra context" che ha senso montare/smontare on-demand, e l'utente vede comunque il riepilogo completo dietro il dialog (continuità percepita). Aggiungere uno step costringerebbe a ricaricare la pagina e a gestire un'altra transizione di stato del wizard.

5.3 Validator minimale

Abbiamo proposto all'utente quattro possibili regole per la validazione "hard": (a) solo anagrafiche + esito + meteo; (b) almeno una prescrizione se l'esito è non conforme; (c) almeno una foto sempre; (d) temperatura obbligatoria. La scelta è ricaduta su (a). Razionale: il vincolo legale è la firma del CSE, le checklist/prescrizioni/foto sono materiale di supporto opzionale; aggiungere vincoli stringenti rischia di bloccare casi legittimi.

5.4 Anno = anno corrente UTC, non anno del verbale

Una bozza creata il 28 dicembre ma firmata il 5 gennaio prende il numero del nuovo anno, non quello vecchio. Razionale: il "registro dei verbali" è organizzato per anno di firma, perché è la firma che dà valore legale; una bozza è solo un work-in-progress.

5.5 EventoAuditTipo.Firma = 3

Per la riga di audit della firma abbiamo aggiunto un nuovo valore all'enum (`Firma = 3`) invece di riusare il generico `TransizioneStato = 1`. Razionale: rendere esplicito e ricercabile il momento della firma nelle query future ("quando è stato firmato l'ultimo verbale di questo cantiere?"). Aggiungere un

valore all’enum non richiede migration perché la colonna in DB è `tinyint` senza check constraint: i vecchi valori 0/1/2 restano validi, 3 è solo un nuovo dominio possibile.

6. Il bug emerso durante il test e come l'abbiamo fixato

Durante il primo smoke test live della firma è emerso un bug **non legato alla firma** ma che si manifestava nel flusso: dopo aver creato una nuova bozza e tentato di scattare una foto nello step 9, il bottone "Scatta" non allegava la foto. L'utente, uscendo dalla bozza e rientrando, vedeva il bottone funzionare correttamente.

La diagnosi

Il log del server raccontava una storia diversa da quella che si vedeva nella UI:

```
Microsoft.Data.SqlClient.SqlException: Violazione del vincolo PRIMARY KEY
'PK_PrescrizioneCse'. Valore della chiave duplicata: 9741291f-...
   at VerbaleRepository.ReplacePrescrizioniAsync
   at VerbaleWizard.HandleStep8SubmitAsync

fail: Microsoft.AspNetCore.Components.Server.Circuits.CircuitHost
   Unhandled exception in circuit 'xvnxqjGLxcHX...'
```

L'errore stava nello **step 8 (prescrizioni)**, non nello step 9. Quando l'utente premeva "Salva e prosegui" dopo aver compilato le prescrizioni, un'eccezione non gestita su `PrescrizioneCse` faceva **cadere il circuito SignalR**. Da quel momento, la UI del browser continuava a renderizzare ma non era più collegata al server: ogni click sembrava "non fare nulla".

Lo scatta foto era una vittima collaterale.

La causa primaria

Quando l'utente preme il bottone "Salva e prosegui" mentre è ancora dentro il TextField di una prescrizione, succedono **due cose contemporaneamente**:

1. Il TextField perde il focus → triggera `OnBlur` → chiama `AutoSaveAsync`.
2. Il bottone riceve `OnClick` → chiama `HandleSubmitAsync`.

Entrambi finiscono per chiamare `VerbaleManager.UpdatePrescrizioniAsync` con la stessa lista. In alcuni interleaving temporali, la lista normalizzata arrivava al repository con **due righe aventi lo stesso Id**, e il batch INSERT esplodeva con `PRIMARY KEY violation`.

Il fix in tre punti

(1) Dedup difensiva nel manager. Anche se la UI dovesse passare una lista con duplicati, il manager genera un Guid nuovo per le occorrenze successive:

```
var seen = new HashSet<Guid>();
foreach (var r in rows) {
    if (string.IsNullOrWhiteSpace(r.Testo)) continue;
    var id = r.Id;
    if (id == Guid.Empty || !seen.Add(id)) {
        id = Guid.CreateVersion7();
        seen.Add(id);
    }
    // crea PrescrizioneCse con id univoco
}
```

(2) Lock nello step 8. Un `SemaphoreSlim(1, 1)` serializza AutoSave e Submit: il submit attende che eventuali AutoSave pendenti finiscano prima di invocare il manager.

(3) Try/catch nel parent. `VerbaleWizard.HandleStep8SubmitAsync` ha ora un `try/catch` che mostra uno `Snackbar` di errore invece di lasciare propagare l'eccezione e far morire il circuito. Se in futuro un altro path produce un errore non gestito, l'utente vede un messaggio comprensibile e l'app resta operativa.

Lezione appresa

Il bug ha mostrato che **un'eccezione non gestita in un event handler Blazor Server è catastrofica** per l'esperienza utente: il circuito muore e la UI continua a renderizzare ma non risponde più. Da ora in poi, ogni event handler dei wizard ha o avrà un `try/catch` che converte le eccezioni in feedback visivo (snackbar/alert) senza lasciar cadere il circuito.

7. Come è stato testato

Test automatici

Sono stati aggiunti **11 nuovi test** (totale ora a 31, tutti verdi):

- `VerbaleValidatorTests` (5 test, unit puro, niente DB): verbale completo è valido; verbale senza esito segnala errore; senza meteo idem; senza anagrafiche segnala 7 errori; verbale firmabile anche senza temperatura/prescrizioni/foto (per blindare la regola "vincolo minimale").
- `VerbaleRepositoryTests` **firma** (3 test, integrazione su SQL Server locale): firmare una bozza assegna `Numero = MAX+1` e sposta lo stato a `FirmatoCse`, scrive le righe `Firma` e `VerbaleAudit`; due verbali firmati consecutivamente nello stesso anno ricevono numeri progressivi; provare a firmare un verbale già firmato lancia `InvalidOperationException`.
- `VerbaleManagerPrescrizioniTests` (3 test, unit con fake repository): la dedup del manager funziona — due `PrescrizioneCse` con stesso Id vengono normalizzate in due Id distinti; righe con testo vuoto/whitespace vengono scartate; Id `Guid.Empty` viene rimpiazzato da uno nuovo.

Test live (smoke)

Dopo i test automatici, un test end-to-end nel browser: login, creazione bozza, compilazione dei 10 step, firma, verifica nella home. Verifiche di coerenza dei dati post-firma:

```sql
-- Verbale firmato con numero progressivo
SELECT Numero, Anno, Stato FROM Verbale WHERE Stato > 0 ORDER BY UpdatedAt DESC;
-- Riga Firma associata
SELECT Tipo, NomeFirmatario, ImmagineFirmaPath FROM Firma WHERE VerbaleId = @id;
-- Audit della firma
SELECT EventoTipo, Note FROM VerbaleAudit WHERE VerbaleId = @id AND EventoTipo = 3;
```

E sul filesystem:

```bash
ls App_Data/uploads/firme/{verbaleId}/cse.png  # ~17-21 KB tipicamente
```

8. Bonus: Guid v7

Dopo aver chiuso B.10 abbiamo fatto un piccolo sweep su tutto il repository: **tutte le 95 occorrenze di `Guid.NewGuid()` sono state sostituite con `Guid.CreateVersion7()`**. La differenza: i Guid v4 (`NewGuid`) sono interamente casuali, i Guid v7 (`CreateVersion7`) hanno i primi 48 bit valorizzati con il timestamp Unix in millisecondi.

Esempio. Due Guid v4 generati in successione:

```
c8a3f10b-7d52-4e1f-9b3a-2f8e6d4a1c09
1f4d822a-3b67-4900-bd14-71e2c9a05e8b
```

Due Guid v7 generati in successione:

```
0192ab8d-7c40-7f1e-9b3a-2f8e6d4a1c09  <- t = 2026-05-13 13:37:42.144
0192ab8d-7c41-7a82-8f23-71e2c9a05e8b  <- t = 2026-05-13 13:37:42.145
```

I primi 8 caratteri sono comuni perché generati a pochi millisecondi di distanza. Ordinati alfabeticamente, i v7 mantengono l'ordine cronologico di generazione.

Perché conta. SQL Server organizza fisicamente le righe sul disco secondo il *clustered index*, che di default è la primary key. Con i Guid v4 random, ogni `INSERT` finiva in una pagina sparsa nel mezzo dell'indice, causando *page splits* e frammentazione. Con i Guid v7 timestamp-ordered, ogni `INSERT` va in coda all'indice — stesso comportamento di un `int IDENTITY`. Niente frammentazione, I/O sequenziale.

Drop-in replacement: stesso tipo C# (`System.Guid`), stesso tipo colonna SQL (`uniqueidentifier`). Le righe esistenti col v4 random restano valide e convivono senza migration. Zero impatto sulla compilazione, zero test rotti.

9. Glossario tecnico

Blazor Server. Modalità di rendering Blazor in cui i componenti girano sul server e comunicano col browser via WebSocket (SignalR). Ogni utente ha un *circuito* sul server che mantiene lo stato.

Circuito SignalR. La connessione persistente tra browser e server in Blazor Server. Se cade (eccezione non gestita, network), l'UI sul browser continua a renderizzare ma non risponde più ai click finché l'utente non ricarica la pagina.

Clustered index. In SQL Server, l'indice fisico secondo cui le righe della tabella sono ordinate sul disco. Di default coincide con la primary key. Conta perché determina l'I/O pattern degli INSERT.

Dapper. Micro-ORM open source per .NET. Più leggero di Entity Framework Core: scriviamo le query SQL a mano, Dapper si occupa solo del mapping risultato → oggetto C#.

ES module. Modulo JavaScript con sintassi `import` / `export`. Caricabile dinamicamente con `await import('/percorso/modulo.js')`.

Idempotenza. Proprietà di un'operazione che, se eseguita più volte con gli stessi parametri, produce lo stesso effetto della prima esecuzione. Importante in HTTP/distributed systems per il retry safe.

JS interop. Meccanismo Blazor per chiamare codice JavaScript dal C# (e viceversa). In Blazor Server le chiamate passano per il circuito SignalR.

Lock pessimistico. Tecnica di concorrenza in cui una transazione richiede esplicitamente al DB di tenere bloccata una riga (o un range) finché non finisce. Le hint `UPDLOCK + HOLDLOCK` di SQL Server fanno esattamente questo.

Minimal API. Pattern ASP.NET Core per definire endpoint HTTP in poche righe direttamente in `Program.cs`, senza Controller.

Race condition. Bug in cui il risultato di un'operazione dipende dall'ordine non controllato di esecuzione di operazioni concorrenti. Tipicamente difficile da riprodurre perché il timing varia.

Repository pattern. Astrazione che separa la logica di accesso ai dati dalla logica di business. In questo progetto: un `Repository` per ogni tabella/aggregate, un `Manager` per ogni `Repository`.

SemaphoreSlim. Primitiva di sincronizzazione .NET per limitare l'accesso concorrente a una risorsa. `new SemaphoreSlim(1, 1)` si comporta come un mutex (uno alla volta).

Transazione. Unità atomica di lavoro su database: o tutte le operazioni dentro la transazione vanno a buon fine, o nessuna viene persistita (rollback).

WeakMap (JS). Struttura dati JavaScript che mappa chiavi (oggetti) a valori, senza impedire la garbage collection delle chiavi quando non sono più referenziate altrove.